Constraint Satisfaction

Constraint Satisfaction

- Constraint satisfaction is a search procedure that operates in a space of constraint sets.
- The initial state contains the constraints that are originally given in the problem description.
- A goal state is any state that has been constrained "enough" where "enough" must be defined for each problem.
- e.g. : In cryptarithematic "enough" means that every number is assigned with a unique numeric value.

Constraint Satisfaction

- Amount of search can be reduced.
- A strait forward search procedure to solve a crypt arithmatic problem might operate in a state space of partial solutions in which letters are assigned particular numbers as their values.
- A depth first control scheme could then follow a path of assignments until either a solution or an inconsistency is discovered.
- In constraint satisfaction approach avoids making wild guess on particular assignments of numbers to letters until it has to.
- .

Crypt arithmetic problem

The initial set of constraints which says that each number may correspond to only one letter and that the sum of the digits must be as they are given in the problem, is first augmented to include restrictions that can be inferred from the rules of arithmetic.

Although searching may still required , the number of allowable guesses is reduced and so the degree of search is curtailed

Constraint Satisfaction

SEND
+MORE

MONEY

Initial State

M=1
S=8 OR 9
O=0 OR 1 → O=0
N=E OR E+1 →
N=E+1
C2=1
N+R > 8
E<> 9

E=2

N=3
R=8 OR 9
2+D =Y OR
2+D=10+Y

C1=0

2+D=Y
N+R=10+E
R=9
S=8

C1=1

2+D=10+Y
D=8+Y
D=8 OR

D=8

Y=0
Conflict

D=9

Y=1
Conflict

Constraint Satisfaction

- Constraint satisfaction is a two step process:
- Step-1
  - Constraints are discovered and propagated as far as possible throughout the system.
  - Then if there is still not a solution, search begins.
  - Step-2
  - A guess about something is made and added as a new constraint.
  - Propagation can still occur with this new constraint and so forth.

Constraint Satisfaction

- The first step propagation arises from the fact that there are usually dependencies among the constraints.
- These dependencies occur because many constraints involve more than one object and more than one objects participate in more than one constraints.
- E.g. :
  - Assume we start with one constraint $N=E+1$
  - Then if we added the constraint $N=3$ we could propagate that to get a stronger constraint on E, $E=2$.

Constraint Satisfaction

- Constraint Satisfaction also arises from the presence of inference rules, that allows additional constraints to be inferred from the ones that are given.
- Constraint propagation terminates for 2 reasons.
  1. A contradiction may be detected.
     - If this happens there is no solution consistent with all known constraints.
     - If the contradiction involves only those constraints that were given as part of the problem, then no solution exists.
  2. Propagation has run out of stream and there are no further changes that can be made on the basis on current knowledge.
     1. If this happens and solution has not yet been adequately specified, then search is necessary to get the process moving again.

Constraint Satisfaction

Constraint Satisfaction

- Algorithm :
  1. Propagate available constraints. To do this first set OPEN to the set of all objects that must have values assigned to them in a complete solution. Then do until an inconsistency is detected or until OPEN is empty:
     a) Select an object OB from OPEN. Strengthen as much as possible the set of constraints that apply to OB.
     b) If this set is different from the set that was assigned the last time OB was examined or if this is the first time OB has been examined, then add to OPEN all objects that share any constraints with OB.
     c) Remove OB from OPEN.

Constraint Satisfaction

2. If the union of the constraints discovered above defines a solution, then quit and report the solution.
3. If the union of the constraints discovered above defines a contradiction , then return failure.
4. If neither of the above occurs , then it is necessary , to make a guess at something in order to proceed To do this , loop until a solution is found or all possible solutions have been eliminated.
   a) Select an object whose value is not yet determined and select a way of strengthening the constraints on that object
   b) Recursively invoke constraint satisfaction with the current set of constraints augmented by the strengthening constraint just selected.

Means-ends analysis

- Involves detection of difference between current state and goal state
- Once difference identified, an operator to reduce the difference must be found
- But perhaps operator cannot be applied to current state
- Subproblem of getting to state where operator can be applied
- Operator may not result in goal state
- Second subproblem of getting from new state to goal state

MEA

- MEA process applied recursively
- Each rule (operator) has
- LHS preconditions and RHS aspects of problem state changed.
- Difference table of rules and differences they can reduce.

MEA

- MEA process applied recursively
- Each rule (operator) has
- LHS preconditions and RHS aspects of problem state changed.
- Difference table of rules and differences they can reduce.

Move desk with 2 things on it to new room

	Push	Carry	Walk	Pickup	Putdown	Place
Move object	*	*				
Move robot			*			
Clear object				*		
Get object on object						*
Get arm empty					*	*
Be holding object				*		

Household robot domain

Initial condition	Final condition
at(robot,x)	at(robot,z)
at(table,y)	at(table,z)
On(block1,table)	On(block1,table)
on(block2,block1)	on(block2,block1)

Operator	Preconditions	Results
PUSH (obj, loc)	at(robot,obj) &large (obj) &clear (obj) & arm empty	at(obj, loc) & at (robot, loc)
CARRY (obj, loc)	at(robot, obj) &Small (obj)	at(obj, loc) &at(robot, loc)
WALK(loc)	none	At(robot, loc)
PICKUP(obj)	At(robot, obj)	Holding(obj)
PUTDOWN(obj)	Holding(obj)	Not holding (obj)
PLACE(obj1, obj2)	At(robot,obj2) & holding (obj1)	on(obj1, obj2)

Means-Ends Analysis (Algorithm)

- 1. Compare CURRENT to GOAL. If no differences, return.
- 2. Otherwise select most important difference and reduce it by doing the following until success or failure is indicated.

(a) Select an as yet untried operator O that is applicable to the current difference. If there are no such operators then signal failure.

(b) Attempt to apply O to the current state. Generate descriptions of two states O-START a state in which O's preconditions are satisfied and O-RESULT, the state that would result if O were applied in O-START.

(c) If (FIRST-PART MEA (CURRENT,O-START) AND (LAST-PART MEA (O-RESULT, GOAL) are successful then signal success.